Expresiones regulares con Python

Apunte didáctico extendido para la librería re

Propósito. Aprender a diseñar expresiones regulares desde búsquedas sencillas hasta patrones capaces de localizar información real con variantes de escritura. Cada metacarácter se introduce antes de utilizarse y los patrones largos se dividen en bloques para que puedan reconstruirse y modificarse con criterio.

Una expresión regular es una descripción compacta de un conjunto de textos posibles. No conviene memorizar patrones completos. Es más útil aprender a formular pequeñas reglas: qué caracteres son obligatorios, cuáles pueden cambiar, cuántas veces pueden aparecer, qué parte es opcional y qué variantes deben aceptarse. A partir de esas reglas se construye la expresión.

Convención de escritura. En Python se recomienda escribir las regex como cadenas crudas: r'\d+'. Así se ve una sola barra invertida. La barra invertida \ introduce muchas instrucciones de regex; la diagonal / es un carácter diferente.

Ejemplos válidos -> **Ejemplos inválidos** -> **Regla pequeña** -> **Prueba** -> **Nueva regla** -> **Patrón completo**

De una palabra literal a una búsqueda precisa

Si el patrón es r'dato', el motor busca las letras d-a-t-o consecutivas. No sabe todavía si forman una palabra independiente. Por eso puede encontrar la misma secuencia dentro de palabras más largas.

```
import re

texto = "dato datos metadato dato"
print(re.findall(r"dato", texto))
```

Salida:

```
['dato', 'dato', 'dato', 'dato']
```

La frontera de palabra \b

\b no significa espacio. Representa una posición en la que se pasa de un carácter de palabra a uno que no lo es, o viceversa. Las letras, los dígitos y el guion bajo se consideran normalmente caracteres de palabra. Espacios y signos de puntuación no lo son.

Visualización. En 'dato,' hay una frontera después de la o: a la izquierda hay una letra y a la derecha una coma. En 'datos' no hay frontera después de la o, porque la s también es una letra. En 'metadato' tampoco hay frontera antes de la d de 'dato', porque justo antes aparece una a.

Texto	¿\bdato\b coincide?	Motivo
dato	Sí	Inicio antes de d y final después de o.
dato,	Sí	La coma crea frontera después de o.
dato final	Sí	El espacio crea frontera después de o.
datos	No	Después de o hay s; no hay frontera.
metadato	No	Antes de d hay a; no hay frontera.
dato_1	No	_ y 1 siguen siendo caracteres de palabra.

```
import re

texto = "dato datos metadato dato, otro dato."
print(re.findall(r"\bdato\b", texto))
```

Salida:

```
['dato', 'dato', 'dato']
```

La idea general es muy útil: si se busca una palabra completa, \b ayuda a evitar coincidencias parciales dentro de palabras más largas.

Clases de caracteres, incluidas sus versiones negativas

Símbolo	Significado	Ejemplos
`\d`	Un dígito decimal.	0, 4, 9
`\D`	`Un carácter que NO sea \d.`	A, -, espacio
`\w`	Letra, dígito o guion bajo.	A, ñ, 7, _
`\W`	`Un carácter que NO sea \w.`	@, -, espacio
`\s`	Un espacio en blanco: espacio, tabulación o salto.	espacio, tab,
`\S`	`Un carácter que NO sea \s.`	A, 9, @

Símbolo	Significado	Ejemplos
`[ABC]`	Una posición que puede ser A, B o C.	A
`[A-Z]`	Una mayúscula dentro de A-Z.	M
`[a-z]`	Una minúscula dentro de a-z.	m
`[^0-9]`	Una posición que NO sea dígito.	A, -, espacio

Metacaracteres que modifican el patrón

Antes de construir nombres, direcciones o correos conviene conocer los símbolos que aparecerán en ellos. Algunos describen caracteres; otros modifican la cantidad o la posición.

Símbolo	Lectura	Ejemplo
.	Cualquier carácter, salvo salto de línea en el modo normal.	c.sa -> casa, cosa, c1sa
`\.`	Un punto literal.	`Dr\. -> Dr.`
+	Una o más repeticiones.	`\d+ -> 7, 25, 2026`
*	Cero o más repeticiones.	a* -> vacío, a, aa...
?	Cero o una repetición del elemento anterior.	colou?r -> color, colour
`{n}`	Exactamente n repeticiones.	`\d{5}`
`{n,m}`	Entre n y m repeticiones.	`[A-Z]{2,4}`
^	Inicio de cadena o línea según configuración.	`^ABC`
$	Final de cadena o línea según configuración.	`fin$`
|	Una alternativa u otra.	`gato|perro`

El punto . y el punto literal \.

El punto es un metacarácter. Si se escribe directamente, no significa 'un punto tipográfico'; significa 'cualquier carácter'. Para pedir un punto real se antepone una barra invertida: \.

```
import re

texto = "Dr. DrX Dra."

print(re.findall(r"Dr.", texto))
print(re.findall(r"Dr\.", texto))
```

Salida:

```
['Dr.', 'DrX', 'Dra']
['Dr.']
```

En r'Dr.' el último símbolo puede ser cualquier carácter: punto, X o a. En r'Dr\.' el último carácter debe ser literalmente un punto.

Por qué Dr.? no significa lo mismo que Dr\.?

```
import re

texto = "Dr Dr. DrX Dra"

print(re.findall(r"Dr.?", texto))
print(re.findall(r"Dr\.?", texto))
```

Salida:

```
['Dr ', 'Dr.', 'DrX', 'Dra']
['Dr', 'Dr.', 'Dr', 'Dr']
```

En Dr.? el punto sigue significando 'cualquier carácter' y ? vuelve opcional a ese carácter cualquiera. Por eso puede aceptar 'DrX' o consumir el espacio después de 'Dr'. En Dr\.? primero se transforma el punto en literal con \. y después ? lo vuelve opcional. Esa es la forma adecuada cuando se desea aceptar Dr y Dr.

Orden mental. Primero pregunte qué representa el elemento: . significa cualquier carácter, mientras que \. significa punto literal. Después aplique el cuantificador ?: \.? significa 'el punto literal puede aparecer cero o una vez'.

El signo ? comparando re.search y re.findall

re.search() busca la primera coincidencia. re.findall() recupera todas. El cuantificador ? puede verse claramente en ambos comportamientos.

```
import re

texto = "color colour colouur colr"

primera = re.search(r"colou?r", texto)
todas = re.findall(r"colou?r", texto)

print("search:", primera.group())
print("findall:", todas)
```

Salida:

```
search: color
findall: ['color', 'colour']
```

Texto	¿Coincide con colou?r?	Razón
color	Sí	La u opcional no aparece.
colour	Sí	La u opcional aparece una vez.
colouur	No	Hay dos u; ? permite como máximo una.
colr	No	La o anterior a u sigue siendo obligatoria.

Inicio y final: cuándo ^ y $ sí cambian el resultado

Los símbolos ^ y $ no suelen ser necesarios cuando el objetivo es extraer un dato que aparece dentro de un texto largo. Sí son importantes cuando se quiere comprobar que un campo completo tenga exactamente un formato y no contenga caracteres adicionales.

Caso de extracción: normalmente no conviene anclar

```
import re

texto = "El folio asignado fue ABC1234 y quedó vigente."

print(re.search(r"ABC\d{4}", texto).group())
print(re.search(r"^ABC\d{4}$", texto))
```

Salida:

```
ABC1234
None
```

Aquí buscamos un dato dentro de una oración, por lo que usar ^ y $ impediría encontrarlo. El texto completo no empieza con ABC ni termina después de los cuatro dígitos.

Caso de validación: aquí sí evitan falsos positivos

```
import re
```

```
campos = [
    "ABC1234",
    "XABC1234",
    "ABC1234X",
    "folio ABC1234"
]

for t in campos:
    sin_anclas = bool(re.search(r"ABC\d{4}", t))
    con_anclas = bool(re.search(r"^ABC\d{4}$", t))
    print(t, sin_anclas, con_anclas)
```

Salida:

```
ABC1234 True True
XABC1234 True False
ABC1234X True False
folio ABC1234 True False
```

Sin anclas, re.search solo pregunta si existe el fragmento correcto en algún lugar. Con ^ y $, se exige que todo el contenido sea exactamente ABC seguido de cuatro dígitos. Por eso son útiles al validar un campo de formulario, una clave o un identificador completo.

Regla práctica. Para extraer dentro de párrafos, normalmente deje libertad antes y después. Para validar que un campo completo no contenga nada extra, considere ^ al principio y $ al final.

Alternativas con | antes de introducir grupos

El operador | se lee 'o'. Permite aceptar una alternativa o la otra. Si las alternativas son completas y no existe una parte común que deba continuar después, puede utilizarse sin paréntesis.

Ejemplo: animales

```
import re

texto = "gato perro conejo gato"
print(re.findall(r"gato|perro", texto))
```

Salida:

```
['gato', 'perro', 'gato']
```

Ejemplo: extensiones de archivo

```
import re

texto = "datos.csv informe.pdf imagen.png notas.txt"
print(re.findall(r"csv|pdf|png", texto))
```

Salida:

```
['csv', 'pdf', 'png']
```

Ejemplo: dos palabras completas

```
import re

texto = "El pago puede hacerse en efectivo o transferencia."
print(re.findall(r"efectivo|transferencia", texto))
```

Salida:

```
['efectivo', 'transferencia']
```

En estos casos, | ya expresa correctamente la intención: una opción completa u otra. Los paréntesis todavía no aportan nada necesario.

Paréntesis: agrupar cambia el alcance de una regla

Los paréntesis son importantes cuando una parte posterior del patrón debe aplicarse a varias alternativas, cuando se quiere repetir un bloque completo o cuando se desea recuperar una parte específica de la coincidencia.

Sin agrupar: el alcance puede ser distinto del que imaginamos

```
import re

texto = "gato gatos perro perros"

print(re.findall(r"gato|perros", texto))
```

Salida:

```
['gato', 'gato', 'perros']
```

Ese patrón significa literalmente 'gato O perros'. La s no se aplica a gato ni a perro de forma general.

Agrupar para aplicar una regla común: primero con paréntesis normales

Antes de introducir (?:...), conviene resolver el problema usando únicamente los paréntesis normales (...). Así se ve con claridad qué aporta el agrupamiento y, después, qué problema adicional resuelve el grupo no capturante.

Queremos reconocer cuatro posibilidades: gato, gatos, perro y perros. La parte variable está en dos lugares: primero elegimos entre las raíces gato y perro; después decidimos si aparece o no la s del plural.

Primer intento: (gato|perro)s?

```
import re

texto = "gato gatos perro perros"

patron = r"(gato|perro)s?"
print(re.findall(patron, texto))
```

Salida:

```
['gato', 'gato', 'perro', 'perro']
```

El patrón sí reconoce las cuatro palabras. Sin embargo, la salida de findall no muestra gatos ni perros completos. Devuelve gato, gato, perro, perro. La razón no está en el operador | ni en el signo ?: está en los paréntesis normales (...), porque en Python esos paréntesis crean un grupo de captura.

(gato | perro)	->	s?	->	coincidencia completa

La parte (gato|perro) significa: elige una de las dos raíces y guárdala como grupo capturado. Después, s? significa: permite una s o ninguna. Por eso el patrón completo puede reconocer 'gatos', pero findall devuelve el contenido del grupo capturado: 'gato'.

Idea importante. Los paréntesis normales tienen dos funciones al mismo tiempo: 1) agrupan varias piezas para que una regla posterior pueda actuar sobre todo el bloque y 2) capturan lo que coincidió dentro del grupo. Muchas veces necesitamos la primera función, pero no la segunda.

Entonces aparece (?:...): agrupar sin capturar

La sintaxis (?:...) se llama grupo no capturante. Los dos puntos no crean una nueva alternativa y no sustituyen al operador |. Su función es decirle a Python: 'trata este bloque como una unidad, pero no lo guardes como un grupo capturado'.

Mismo patrón lógico, pero sin captura

```
import re

texto = "gato gatos perro perros"

patron = r"(?:gato|perro)s?"
print(re.findall(patron, texto))
```

Salida:

```
['gato', 'gatos', 'perro', 'perros']
```

Ahora findall devuelve las palabras completas. La alternancia sigue estando creada por |. Los paréntesis siguen agrupando gato y perro para que s? se aplique a cualquiera de las dos opciones. Lo único que cambia con ?: es que ese grupo deja de formar parte de la salida como captura.

Parte	Función
(?:	abre un grupo no capturante
gato	primera alternativa
|	significa “o”
perro	segunda alternativa
)	cierra el grupo
s?	la s puede aparecer cero o una vez

La misma idea vista sin ?, con ? y con una s obligatoria

```
import re

textos = ["gato", "gatos", "perro", "perros"]

for t in textos:
    a = bool(re.search(r"^(?:gato|perro)$", t))
    b = bool(re.search(r"^(?:gato|perro)s?$", t))
    c = bool(re.search(r"^(?:gato|perro)s$", t))
    print(t, a, b, c)
```

Salida:

```
gato True True False
gatos False True True
perro True True False
perros False True True
```

La primera columna solo acepta singular porque después del grupo no existe ninguna s en el patrón. La segunda acepta singular y plural porque s? hace opcional la s. La tercera exige plural porque la s está escrita sin ? y, por tanto, es obligatoria.

Otro uso del grupo: repetir un bloque completo con +

El signo + significa 'una o más repeticiones del elemento inmediatamente anterior'. Si el elemento anterior es una letra, repite esa letra. Si el elemento anterior es un grupo, repite todo el grupo.

Sin + frente a + aplicado al grupo

```
import re

for t in ["ha", "haha", "hahaha", "haaa", "h"]:
    sin_mas = bool(re.search(r"^ha$", t))
    con_mas = bool(re.search(r"^(?:ha)+$", t))
    print(t, sin_mas, con_mas)
```

Salida:

```
ha True True
haha False True
hahaha False True
haaa False False
h False False
```

Sin +, el patrón ^ha$ exige una única aparición exacta de ha. Con (?:ha)+, el grupo ha puede repetirse una o más veces: ha, haha, hahaha, etc. 'haaa' sigue siendo inválido porque no puede dividirse en repeticiones completas del bloque ha.

Qué ocurre si quitamos los paréntesis y dejamos ha+

```
import re

for t in ["ha", "haa", "haaa", "haha"]:
    print(t, bool(re.search(r"^ha+$", t)))
```

Salida:

```
ha True
haa True
haaa True
haha False
```

Aquí + ya no se aplica a 'ha'. Se aplica únicamente a la a, porque es el elemento inmediatamente anterior. Por eso ha+ significa h seguido de una o más letras a: ha, haa, haaa... y no acepta haha.

Patrón	Unidad que se repite	Acepta	No acepta
`^ha$`	ninguna repetición adicional	ha	haha, haaa
`^ha+$`	solo la letra a	ha, haa, haaa	haha
`^(ha)+$`	todo el grupo ha	ha, haha, hahaha	haa, haaa
`^(?:ha)+$`	todo el grupo ha, sin captura	ha, haha, hahaha	haa, haaa

Comparación directa: sin paréntesis y con paréntesis

Los siguientes miniejemplos muestran un mismo problema antes y después de agrupar. El objetivo es observar exactamente qué regla termina afectando a qué parte.

Objetivo	Sin agrupar	Interpretación real	Agrupado	Interpretación correcta
gato/perro singular o plural	gato|perros?	'gato' O 'perro' con s opcional	`(?:gato|perro)s?`	elegir raíz y luego plural opcional
Dr. o Dra. con punto	`Dr|Dra\.`	'Dr' O 'Dra.'	`(?:Dr|Dra)\.`	elegir título y luego punto para ambos
ab o cd repetidos	ab|cd+	'ab' O 'c' + varias d	`(?:ab|cd)+`	repetir cualquiera de los bloques completos
lunes o martes opcionalmente con s	lunes|martess?	'lunes' O 'martes' + s opcional	`(?:lunes|martes)s?`	regla s? posterior aplicada a ambas alternativas
PNG/JPG con extensión .bak opcional	`png|jpg\.bak?`	'png' O patrón parcial sobre jpg	`(?:png|jpg)(?:\.bak)?`	elige extensión y luego bloque .bak opcional
casa o piso seguido de número	`casa|piso\d+`	'casa' O 'piso' con uno o más dígitos	`(?:casa|piso)\d+`	ambas palabras deben ir seguidas de número

Paréntesis de captura (...) frente a grupos no capturantes (?:...)

Ahora que ya vimos por qué agrupar cambia el alcance de ? y +, podemos separar con claridad dos ideas diferentes: agrupar y capturar. Ambos (...) y (?:...) agrupan. La diferencia es que (...) además guarda el contenido del grupo; (?:...) no lo guarda.

Lectura visual. (...) = AGRUPAR + CAPTURAR. (?:...) = AGRUPAR SIN CAPTURAR. El operador | conserva exactamente el mismo significado dentro de ambos: sigue significando 'o'.

Primero, solo paréntesis normales (...)

```
import re

texto = "gato gatos perro perros"
```

```
print(re.findall(r"(gato|perro)s?", texto))
```

Salida:

```
['gato', 'gato', 'perro', 'perro']
```

El patrón completo reconoce singular y plural, pero findall devuelve el grupo capturado. Esta conducta es útil si precisamente queremos extraer solo la raíz. De hecho, podría aprovecharse para convertir gato/gatos en gato y perro/perros en perro.

Después, el mismo patrón con (?:...)

```
import re

texto = "gato gatos perro perros"

print(re.findall(r"(?:gato|perro)s?", texto))
```

Salida:

```
['gato', 'gatos', 'perro', 'perros']
```

Como ya no hay grupo capturado, findall devuelve la coincidencia completa. Esta suele ser la forma más conveniente cuando los paréntesis solo se agregaron para aplicar una regla común.

Retomando Dr. y Dra. paso a paso, mostrando la salida

Este ejemplo permite separar con claridad alternancia, agrupamiento, captura y opcionalidad del punto.

A) Dos alternativas completas: el punto es obligatorio

```
import re

texto = "Dr. Carlos, Dra. Ana, Dr Pedro, Dra Lucía"

patron1 = r"Dr\.|Dra\."
print(re.findall(patron1, texto))
```

Salida:

```
['Dr.', 'Dra.']
```

Aquí | conecta dos patrones completos. No se necesitan paréntesis porque no existe una regla posterior que deba afectar a ambas alternativas.

B) Agrupar con (...): el punto sigue siendo obligatorio

```
patron2 = r"(Dr|Dra)\."
print(re.findall(patron2, texto))
```

Salida:

```
['Dr', 'Dra']
```

Ahora evitamos repetir la estructura, pero creamos una captura. Por ello findall devuelve Dr y Dra, sin el punto, aunque el punto sí fue necesario para que la coincidencia ocurriera.

C) Agrupar con (?:...): el punto sigue siendo obligatorio

```
patron3 = r"(?:Dr|Dra)\."
print(re.findall(patron3, texto))
```

Salida:

```
['Dr.', 'Dra.']
```

La lógica de coincidencia es igual a la anterior, pero el grupo ya no se captura. Por eso findall devuelve el título completo.

D) Ahora el punto es opcional con \.?

```
patron4 = r"(?:Dr|Dra)\.?"
print(re.findall(patron4, texto))
```

Salida:

```
['Dr.', 'Dra.', 'Dr', 'Dra']
```

El ? está después de \., así que únicamente el punto es opcional. Dr y Dra siguen siendo obligatorios como alternativas. Esta versión acepta títulos escritos con punto o sin punto.

Tres comparativas adicionales: (...) contra (?:...)

En cada caso el patrón reconoce el mismo texto; lo que cambia es qué devuelve findall cuando existen grupos capturados.

Ejemplo 1: fechas

```
import re

texto = "2025-10-01 2026/11/22"

print(re.findall(r"(\d{4})[-/](\d{2})[-/](\d{2})", texto))
print(re.findall(r"(?:\d{4})[-/](?:\d{2})[-/](?:\d{2})", texto))
```

Salida:

```
[('2025', '10', '01'), ('2026', '11', '22')]
['2025-10-01', '2026/11/22']
```

Con (...) se capturan por separado año, mes y día, lo cual es útil si se necesitan esos componentes. Con (?:...) solo se agrupa y findall devuelve cada fecha completa.

Ejemplo 2: extensiones

```
import re

texto = "archivo.csv foto.png informe.pdf"

print(re.findall(r"archivo\.(csv|pdf|png)", texto))
print(re.findall(r"archivo\.(?:csv|pdf|png)", texto))
```

Salida:

```
['csv']
['archivo.csv']
```

Si capturamos (csv|pdf|png), findall devuelve solo la extensión. Si no necesitamos esa pieza por separado, (?:csv|pdf|png) evita la captura y conserva la coincidencia completa.

Ejemplo 3: prefijos de cantidades

```
import re

texto = "MXN 850 $1200 MXN 75"

print(re.findall(r"(MXN|\$)\s?\d+", texto))
print(re.findall(r"(?:MXN|\$)\s?\d+", texto))
```

Salida:

```
['MXN', '$', 'MXN']
['MXN 850', '$1200', 'MXN 75']
```

La primera versión captura únicamente el prefijo. La segunda usa el grupo solo para elegir entre MXN o $, así que findall devuelve la cantidad completa.

Qué cambia cuando aparece ? después de un grupo

El lugar donde se escribe ? determina qué se vuelve opcional. Si aparece dentro del grupo, afecta al elemento anterior dentro del grupo. Si aparece después del paréntesis de cierre, vuelve opcional a todo el grupo.

```
import re

textos = ["Carlos", "Dr Carlos", "Dr. Carlos"]

patron_sin_opcional = r"^(?:Dr\.\s+)Carlos$"
patron_grupo_opcional = r"^(?:Dr\.\s+)?Carlos$"
patron_punto_opcional = r"^(?:Dr\.?\s+)Carlos$"

for t in textos:
    print(
        t,
        bool(re.search(patron_sin_opcional, t)),
        bool(re.search(patron_grupo_opcional, t)),
```

```
        bool(re.search(patron_punto_opcional, t))
    )
```

Salida:

```
Carlos False True False
Dr Carlos False False True
Dr. Carlos True True True
```

Sin ?, el título completo Dr. + espacio es obligatorio. Con ? después del grupo, todo el bloque del título se vuelve opcional y por eso también se acepta Carlos sin título. Con ? únicamente después de \., el título Dr sigue siendo obligatorio, pero el punto puede faltar.

Más miniejemplos para decidir qué usar

Necesidad	Patrón	¿Captura?	Comentario
gato o perro	gato|perro	No	No hace falta agrupar.
gato/perro + plural opcional	(gato|perro)s?	Sí	Útil si quiero recuperar solo la raíz.
gato/perro + plural opcional	(?:gato|perro)s?	No	Útil si quiero la palabra completa.
Dr. o Dra. punto obligatorio	(Dr|Dra)\.	Sí	findall devuelve Dr/Dra.
Dr. o Dra. punto obligatorio	(?:Dr|Dra)\.	No	findall devuelve Dr./Dra. completos.
Dr/Dra con punto opcional	(?:Dr|Dra)\.?	No	Acepta con o sin punto.
bloque ha repetido	(ha)+	Sí	Agrupa y captura; + repite el bloque.
bloque ha repetido	(?:ha)+	No	Misma coincidencia, sin captura.
fecha y componentes separados	(\d{2})/(\d{2})/(\d{4})	Sí	Aquí capturar es deseable.
una extensión entre varias	(?:csv|pdf|png)	No	Solo necesitamos organizar alternativas.

Resumen conceptual. | elige entre alternativas. (...) delimita una unidad y además captura. (?:...) delimita la misma unidad sin capturar. ? vuelve opcional al elemento o grupo inmediatamente anterior. + repite una o más veces al elemento o grupo inmediatamente anterior. Antes de escribir un patrón, conviene identificar exactamente a qué unidad debe afectar cada cuantificador.

El espacio también forma parte del patrón: \s, \s+ y \s*

Cuando dos palabras deben aparecer separadas, la regex necesita describir esa separación. \b solo comprueba una frontera, pero no avanza sobre el espacio. \s consume un carácter de espacio en blanco. \s+ consume uno o más. \s* permite cero o más.

```
import re

texto = "Ana María"

print(bool(re.search(r"Ana\bMaría", texto)))
print(bool(re.search(r"Ana\s+María", texto)))
```

Salida:

```
False
True
```

Después de encontrar Ana, \b verifica que termina una palabra, pero el motor sigue situado antes del espacio. El siguiente elemento del patrón es M y no coincide con el espacio. En cambio, \s+ consume el espacio y deja al motor listo para buscar María.

Patrón	Qué permite	Ejemplo
`Ana\sMaría`	Exactamente un carácter de espacio en blanco	Ana María
`Ana\s+María`	Uno o más espacios/tabulaciones	Ana María
`Ana\s*María`	Cero o más	AnaMaría o Ana María
`Ana\b`	Solo comprobar frontera	Ana, / Ana

Nombres de personas: construir primero la palabra y después la secuencia

Antes de buscar un nombre completo, conviene definir qué aspecto tendrá una sola palabra. Para un ejemplo didáctico aceptaremos una inicial mayúscula seguida de una o más minúsculas, incluyendo vocales acentuadas y ñ.

`[A-ZÁÉÍÓÚÑ]`	`[a-záéíóúñ]+`
una inicial mayúscula	una o más minúsculas

```
import re

texto = "Ana carlos María LÓPEZ Pedro"
print(re.findall(r"\b[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+\b", texto))
```

Salida:

```
['Ana', 'María', 'Pedro']
```

LÓPEZ no coincide porque después de la L aparecen otras mayúsculas. Esto no significa que LÓPEZ sea inválido como nombre; significa que nuestra regla actual no contempla palabras completamente en mayúsculas.

Cuatro palabras con aspecto de nombre

`(?:`	`[A-Z...][a-z...]+`	`\s+`	`){3}`	`[A-Z...][a-z...]+`
grupo sin captura	una palabra	separación obligatoria	repetir tres veces	cuarta palabra

```
import re

texto = """
Ana María López García
Carlos Alberto Ruiz Pérez
Ana López
"""

patron = (
    r"(?:[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+\s+){3}"
    r"[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+"
)

print(re.findall(patron, texto))
```

Salida:

```
['Ana María López García', 'Carlos Alberto Ruiz Pérez']
```

Añadir títulos opcionales

```
patron_titulo = r"(?:Dr\.?|Dra\.?|Ing\.?|Mtro\.?|Mtra\.?)?"
```

El grupo reúne varias alternativas de título. El ? colocado después del cierre del grupo hace opcional a todo el título, no solo a su última letra. Dentro de cada alternativa, \.? hace opcional únicamente el punto.

Correo electrónico: dividir un patrón largo en unidades comprensibles

Los correos son útiles para practicar porque combinan clases de caracteres, cuantificadores, punto literal, grupos y repeticiones. El siguiente patrón es práctico para extracción general, no una implementación completa del estándar internacional de correo electrónico.

Primera unidad: la parte local antes de @

[	\w	.+-	]	+
abre clase	letras, dígitos y _	también punto, + y guion	cierra clase	uno o más caracteres

[\w.+-]+ significa que cada posición puede contener un carácter de \w, un punto, un signo + o un guion, y que debe existir al menos un carácter. Los corchetes son necesarios porque queremos definir un conjunto de opciones para una sola posición; el + que está fuera de los corchetes repite esa posición una o más veces.

```
import re

texto = "ana ana.lopez alumno_01 nombre+grupo"
print(re.findall(r"[\w.+-]+", texto))
```

Salida:

```
['ana', 'ana.lopez', 'alumno_01', 'nombre+grupo']
```

Segunda unidad: @ y el primer bloque del dominio

[\w.+-]+	@	[\w-]+
parte local	separador literal	nombre inicial del dominio

@ no es metacarácter en este contexto, por lo que se escribe directamente. [\w-]+ permite letras, números, guion bajo y guion en el primer bloque del dominio.

Tercera unidad: uno o más bloques .dominio

(?:	\.	[\w-]+	)	+
agrupar sin capturar	punto literal	bloque después del punto	fin del grupo	repetir el bloque 1 o más veces

La parte (?:\.[\w-]+)+ es esencial para aceptar tanto example.com como escom.ipn.mx o sub.dominio.org. Si se escribiera solo \.[\w-]+, se permitiría un único bloque después del primer punto. El grupo permite tratar 'punto + bloque' como una unidad, y el + externo repite esa unidad tantas veces como sea necesario.

Patrón completo

[\w.+-]+	@	[\w-]+	(?:\.[\w-]+)+
parte local	@	primer dominio	uno o más bloques .ext

```
import re

texto = """
ana@example.com
ana.lopez@empresa.com
alumno_01@escom.ipn.mx
nombre+grupo@sub.dominio.org
```

```
correo@dominio
"""

patron = r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+"
print(re.findall(patron, texto))
```

Salida:

```
['ana@example.com',
 'ana.lopez@empresa.com',
 'alumno_01@escom.ipn.mx',
 'nombre+grupo@sub.dominio.org']
```

Correo	¿Se encuentra?	Motivo
ana@example.com	Sí	Un bloque .com.
alumno_01@escom.ipn.mx	Sí	El grupo .bloque se repite dos veces.
nombre+grupo@sub.dominio.org	Sí	Parte local ampliada y varios dominios.
correo@dominio	No	Falta al menos un bloque con punto.
@dominio.com	No	Falta la parte local.

Qué pasaría sin corchetes

```
# No equivalente:
patron_mal = r"\w.+-+@..."
```

Sin corchetes, los símbolos dejan de formar un conjunto de opciones. Por ejemplo, . volvería a significar 'cualquier carácter' y + actuaría como cuantificador. Los corchetes [\w.+-] indican que todos esos elementos se interpretan como opciones permitidas para una misma posición.

Direcciones: alternativas, abreviaturas y componentes opcionales

Una dirección puede iniciar con Calle, C., Avenida, Av., Boulevard o Blvd. Después puede contener varias palabras, partículas como de/del/los, número exterior, interior y código postal.

Introducir el grupo de tipos de vía

(?:	Calle	|	C\.	|	Av\.?	| ...	)	\s+
grupo sin captura	opción	o	opción	o	opción	más opciones	fin	espacio después de la vía

```
import re

texto = """
Calle Reforma 120
Av. Universidad 300
Avenida Insurgentes 450
C. Madero 25
Blvd. Central 900
"""

patron = r"\b(?:Calle|C\.|Av\.?|Avenida|Blvd\.?|Boulevard)\s+"
print(re.findall(patron, texto))
```

Salida:

```
['Calle ', 'Av. ', 'Avenida ', 'C. ', 'Blvd. ']
```

Aquí \s+ sí es necesario porque después del tipo de vía esperamos consumir la separación para continuar leyendo el nombre de la calle. \b solo confirmaría que terminó una palabra, pero no avanzaría sobre el espacio.

Dirección con número, interior y código postal opcionales

```
import re

via = r"(?:Calle|C\.|Av\.?|Avenida|Blvd\.?|Boulevard)"
palabra = r"(?:[A-Za-zÁÉÍÓÚÑáéíóúñ.]+|de|del|la|las|los)"
numero = r"(?:No\.?|Núm\.?|#)?\s*\d+[A-Za-z]?"
interior = r"(?:Int\.?|Interior)\s*\d+[A-Za-z]?"
cp = r"(?:C\.?P\.?|CP)\s*\d{5}"

patron = (
    rf"\b{via}\s+"
    + palabra + rf"(?:\s+{palabra}){{0,6}}"
    + rf"(?:\s+{numero})?"
    + rf"(?:\s*,?\s*{interior})?"
    + rf"(?:\s*,?\s*{cp})?"
)

ejemplos = [
    "Av Universidad No. 300, CP 04360",
    "Avenida de los Insurgentes #450 Int. 4, CP 03100",
    "C. Madero 25A, Interior 2B, C.P. 06000"
]

for e in ejemplos:
    m = re.search(patron, e, re.IGNORECASE)
    print(m.group() if m else None)
```

Salida:

```
Av Universidad No. 300, CP 04360
Avenida de los Insurgentes #450 Int. 4, CP 03100
C. Madero 25A, Interior 2B, C.P. 06000
```

La estrategia más clara consiste en dar nombre conceptual a cada subpatrón: via, palabra, numero, interior y cp. Después se ensamblan. Esto facilita modificar una parte sin reescribir toda la expresión.

Familias de palabras, singular/plural y faltas previsibles

Las regex pueden agrupar variantes de una palabra cuando conocemos de antemano qué diferencias queremos tolerar. Esto resulta útil para normalizar vocabulario antes de contar palabras o construir representaciones de texto.

Singular y plural sencillo

```
import re

texto = "cliente clientes producto productos dato datos"

print(re.findall(r"\bclientes?\b", texto))
print(re.findall(r"\bproductos?\b", texto))
print(re.findall(r"\bdatos?\b", texto))
```

Salida:

```
['cliente', 'clientes']
['producto', 'productos']
['dato', 'datos']
```

La s final es opcional. Esta estrategia funciona cuando el plural se forma agregando s sin modificar la raíz.

Raíz común con varias terminaciones: idea de lematización

```
import re

texto = "programar programa programamos programaron programando"
patron = r"\bprogram(?:ar|a|amos|aron|ando)\b"

for forma in re.findall(patron, texto):
    print(forma, "-> programar")
```

Salida:

```
programar -> programar
programa -> programar
```

```
programamos -> programar
programaron -> programar
programando -> programar
```

El patrón conserva la raíz program y enumera terminaciones conocidas. En un vocabulario controlado esto puede utilizarse como lematización manual: varias formas superficiales se transforman en un mismo lema. Para español general se requiere un lematizador más completo porque existen muchas conjugaciones e irregularidades.

Omisión de una letra con ?

```
import re

texto = "disciplina diciplina"
print(re.findall(r"\bdis?ciplina\b", texto))
```

Salida:

```
['disciplina', 'diciplina']
```

La s queda opcional. Así pueden detectarse juntas la forma correcta y una falta concreta en la que la s se omite.

Sustitución de una letra mediante una clase

```
import re

texto = "canción cansion decisión desisión"

print(re.findall(r"\bcan[cs]i[oó]n\b", texto))
print(re.findall(r"\bde[cs]isi[oó]n\b", texto))
```

Salida:

```
['canción', 'cansion']
['decisión', 'desisión']
```

[cs] acepta c o s en esa posición; [oó] acepta la vocal con o sin acento. Se trata de reglas dirigidas a errores conocidos, no de un corrector general.

Usar * para tolerar repeticiones u omisiones controladas

```
import re

texto = "cooperar coperar coooperar"

# Una o más letras o después de c
print(re.findall(r"\bco+perar\b", texto))

# Cero o más letras o después de c
print(re.findall(r"\bco*perar\b", texto))
```

Salida:

```
['cooperar', 'coperar', 'coooperar']
['cooperar', 'coperar', 'coooperar']
```

En este caso + y * producen los mismos resultados con los ejemplos dados porque todos tienen al menos una o. La diferencia aparecería con cperar: co*perar lo permitiría, co+perar no. Esta comparación ayuda a decidir cuánto error se desea tolerar.

Normalizar varias formas con re.sub

```
import re

texto = "La diciplina y la cansion aparecen en el texto."

texto = re.sub(r"\bdis?ciplina\b", "disciplina", texto)
texto = re.sub(r"\bcan[cs]i[oó]n\b", "canción", texto)

print(texto)
```

Salida:

```
La disciplina y la canción aparecen en el texto.
```

Folios, cantidades y montos monetarios

Los documentos de trabajo suelen contener identificadores compuestos por letras y números, cantidades enteras o decimales y montos con separadores de miles. Estos patrones son una de las aplicaciones más directas de las regex.

Folios con estructuras diferentes

Ejemplo	Regex	Lectura
ABC-2026-00451	`\b[A-Z]{3}-\d{4}-\d{5}\b`	3 letras, año, consecutivo.
REQ-IT-0834	`\b[A-Z]{3}-[A-Z]{2}-\d{4}\b`	tipo, área, número.
FOL/2026/971	`\b[A-Z]{3}/\d{4}/\d{3}\b`	prefijo/año/número.
A1B9C25	`\b(?=[A-Z0-9]*[A-Z])(?=[A-Z0-9]*\d)[A-Z0-9]{5,10}\b`	alfanumérico con al menos una letra y un dígito.

Cantidad decimal opcional

`\b`	`\d+`	`(?:`	`\.`	`\d+`	`)?`	`\b`
frontera	parte entera	grupo sin captura	punto decimal	decimales	grupo opcional	frontera

```
import re

texto = "Se reportaron 7 casos, 25.5 horas y 1200 registros."
print(re.findall(r"\b\d+(?:\.\d+)?\b", texto))
```

Salida:

```
['7', '25.5', '1200']
```

Dinero con $, MXN, comas y centavos

`(?:MXN\s*|\$\s*)?`	`(?:\d{1,3}(?:,\d{3})+|\d+)`	`(?:\.\d{2})?`
prefijo opcional	cantidad	centavos opcionales

```
import re

texto = "Pagos: $850, $1,250.50, MXN 3000.00 y $12,000."

patron = (
    r"(?:MXN\s*|\$\s*)?"
    r"(?:\d{1,3}(?:,\d{3})+|\d+)"
    r"(?:\.\d{2})?"
)

print(re.findall(patron, texto))
```

Salida:

```
['$850', '$1,250.50', 'MXN 3000.00', '$12,000']
```

La cantidad principal contiene una alternativa: números con comas agrupadas de tres en tres o números simples sin comas. Los centavos son opcionales, pero cuando aparecen se exigen dos dígitos.

Funciones de re: buscar, recuperar, recorrer y transformar

Función	Uso
re.search()	Devuelve la primera coincidencia encontrada.

Función	Uso
re.findall()	Devuelve todas las coincidencias como lista.
re.finditer()	Recorre todas y conserva posición/grupos.
re.sub()	Sustituye o normaliza coincidencias.
re.split()	Divide un texto usando un patrón.
re.compile()	Prepara un patrón para reutilizarlo.

search y findall en el mismo problema

```
import re

texto = "Folios: ABC-2026-00451, PLN-2026-00007"
patron = r"\b[A-Z]{3}-\d{4}-\d{5}\b"

primero = re.search(patron, texto)
todos = re.findall(patron, texto)

print(primero.group())
print(todos)
```

Salida:

```
ABC-2026-00451
['ABC-2026-00451', 'PLN-2026-00007']
```

finditer cuando importan las posiciones

```
import re

texto = "A2048 B0007 C9512"

for m in re.finditer(r"\b[A-C]\d{4}\b", texto):
    print(m.group(), m.start(), m.end())
```

Salida:

```
A2048 0 5
B0007 6 11
C9512 12 17
```

Los objetos Match aparecen aquí, después de comprender la búsqueda. group() devuelve lo encontrado y start()/end() indican su ubicación.

Condiciones sin consumir texto: lookarounds

Los lookarounds permiten exigir contexto antes o después sin incluirlo en el resultado. Son útiles en patrones avanzados, por ejemplo para extraer números solo cuando están asociados a un símbolo o para exigir tipos de caracteres en cualquier orden.

Sintaxis	Lectura
(?=...)	Después debe existir ...
(?!...)	Después NO debe existir ...
(?<=...)	Antes debe existir ...
(?<!...)	Antes NO debe existir ...

```
import re

texto = "Precio $850, descuento 20, total $1200"
print(re.findall(r"(?<=\$)\d+", texto))
```

Salida:

```
['850', '1200']
```

Letras y números en cualquier orden

```
import re

codigos = ["AB12", "A12B", "12AB", "ABC", "1234"]
patron = r"^(?=.*[A-Z])(?=.*\d)[A-Z0-9]+$"

for c in codigos:
    print(c, bool(re.search(patron, c)))
```

Salida:

```
AB12 True
A12B True
12AB True
ABC False
1234 False
```

(?=.*[A-Z]) exige que exista al menos una mayúscula en algún punto; (?=.*\d) exige al menos un dígito. Como ambas condiciones solo miran y no consumen, el orden puede variar.

Caso integrador: correo laboral con información distribuida

El siguiente mensaje ficticio contiene nombres con y sin título, direcciones en varias formas, fechas, correos, folios, referencias, cantidades y algunos errores de escritura deliberados. El objetivo no es resolverlo con una regex gigantesca, sino mostrar cómo se puede construir un pequeño extractor por categorías.

Texto de entrada

```
Asunto: Seguimiento a incidencia de conciliación y documentación pendiente

Buen día, equipo:

Les comparto el seguimiento de la incidencia que comenzó el 14/10/2026 durante el cierre de
conciliación del proyecto de renovación de terminales. El primer aviso se recibió aproximadamente a las
08:35 horas, cuando la Dra. Ana María López García, responsable del área de Operaciones, informó que
tres movimientos relacionados con el folio OP-2026-00481 no aparecían reflejados en el reporte de
cierre. La Dra. López indicó que el problema había sido identificado en las oficinas ubicadas en Av.
Universidad No. 300, Col. Copilco, C.P. 04360, y solicitó que cualquier actualización se enviara a
ana.lopez@empresa.com y, en copia, a operaciones.mx@empresa.com.

A las 09:10 se revisó el primer lote. El archivo contenía 1,284 registros y una diferencia acumulada de
$18,450.75. En ese momento, el Ing. Carlos Alberto Ruiz Pérez comentó por correo que una parte del
desfase podía estar asociada con la solicitud REQ-IT-0834, levantada el 13-10-2026 para modificar la
regla de redondeo del proceso. Carlos pidió que las pruebas se documentaran y dejó como contacto
carlos.ruiz@soporte.empresa.com. En su mensaje también señaló que el cambio había sido autorizado con
el documento FOL/2026/917, aunque posteriormente se confirmó que el número correcto era FOL/2026/971.

Durante la segunda revisión, realizada el 15 de octubre de 2026, se detectaron 27 operaciones con un
comportamiento distinto al esperado. El monto total de esas operaciones fue MXN 7,320.00. La
información fue enviada a Mtra. Lucía Fernanda Torres Ortiz, quien se encontraba trabajando desde la
sede de Avenida de los Insurgentes #450, Interior 4, CP 03100. Su correo institucional es
lucia.torres@auditoria.org.mx. En el mensaje original aparecía la etiqueta “correoo:” y la dirección
estaba escrita como “Direcion: Avenida de los Insurgentes #450”, por lo que conviene considerar ambas
variantes al realizar la extracción automática.

Más tarde, el 16/10/2026, Pedro Antonio Cruz Díaz informó que el archivo de respaldo contenía un
registro adicional asociado al folio ABC-2026-00512. Pedro no utiliza título profesional en su firma y
aparece únicamente como “Pedro Antonio Cruz Díaz”. Su correo es pedro.cruz@datos.net. Señaló que el
registro adicional correspondía a una devolución por $1,250.50 y que el comprobante se había almacenado
con la referencia DEV-OP-0027. El domicilio del proveedor que emitió la devolución se registró como C.
Madero 25A, Interior 2B, C.P. 06000. En otra versión del documento se utilizó la forma “Calle Madero
No. 25A, Int. 2B, CP 06000”, por lo que ambos formatos deben considerarse equivalentes para fines de
búsqueda.

El 17-10-2026 a las 11:20, la Dra Ana María López García confirmó que, después de reprocesar el lote,
la diferencia se redujo de $18,450.75 a $850.00. Sin embargo, todavía quedaban 4 movimientos
pendientes. En el mismo mensaje escribió “diciplina operativa” en lugar de “disciplina operativa” y
“cansion de respaldo” donde debía decir “canción de respaldo”; estos errores no afectan el contenido
del caso, pero son útiles para probar reglas de normalización orientadas a faltas conocidas.
```

```
Se acordó que la validación final se realizaría el 20/10/2026. Para esa sesión debían participar la
Dra. Ana María López García, el Ing. Carlos Alberto Ruiz Pérez, Mtra. Lucía Fernanda Torres Ortiz y
Pedro Antonio Cruz Díaz. La reunión se programó en Blvd. Central 900, CP 11560, aunque el aviso de
calendario utilizó la variante “Boulevard Central No 900, C.P. 11560”. El acceso al expediente quedó
relacionado con los folios OP-2026-00481, ABC-2026-00512, REQ-IT-0834, DEV-OP-0027 y FOL/2026/971.

El 20 de octubre, a las 16:40, se realizó la revisión final. Se confirmaron 1,284 registros procesados,
1,284 registros conciliados y una diferencia final de $0.00. El equipo acordó conservar el folio
OP-2026-00481 como identificador principal de la incidencia y utilizar ABC-2026-00512 únicamente como
referencia del registro adicional. También se acordó que el documento REQ-IT-0834 permanecería asociado
al cambio de configuración y que FOL/2026/971 funcionaría como evidencia de autorización.

Como actividad posterior, se pidió a Carlos enviar antes del 22/10/2026 una breve descripción de la
modificación aplicada; a Lucía, revisar que los comprobantes estuvieran completos; y a Pedro, confirmar
que la devolución de $1,250.50 estuviera correctamente vinculada con DEV-OP-0027. Ana será la
responsable de cerrar formalmente el incidente. Si surge alguna observación adicional, se deberá
escribir a ana.lopez@empresa.com o a operaciones.mx@empresa.com. Para temas técnicos se utilizará
carlos.ruiz@soporte.empresa.com, mientras que la revisión documental podrá dirigirse a
lucia.torres@auditoria.org.mx.

Finalmente, se dejó constancia de que el incidente no produjo pérdida de información y que los montos
observados correspondieron a diferencias temporales de conciliación. El total de $18,450.75
identificado al inicio fue disminuyendo conforme se reprocesaron las operaciones hasta llegar a $0.00.
El caso se considera cerrado, pero los documentos y correos se conservarán como material de auditoría
para futuras revisiones.

Saludos,
Coordinación de Seguimiento Operativo
```

Separar la extracción por categorías

```
import re

correo = r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+"

fecha = (
    r"\b(?:"
    r"\d{2}[/-]\d{2}[/-]\d{4}"
    r"|"
    r"\d{1,2}\s+de\s+[A-Za-záéíóúñ]+\s+de\s+\d{4}"
    r")\b"
)

folio = (
    r"\b(?:"
    r"[A-Z]{2,4}-\d{4}-\d{4,5}"
    r"|[A-Z]{3}-[A-Z]{2}-\d{4}"
    r"|[A-Z]{3}/\d{4}/\d{3}"
    r")\b"
)

dinero = (
    r"(?:MXN\s*|\$\s*)"
    r"(?:\d{1,3}(?:,\d{3})+|\d+)"
    r"(?:\.\d{2})?"
)

nombre = (
    r"\b(?:Dr\.?|Dra\.?|Ing\.?|Mtra\.?|Mtro\.?)?\s*"
    r"(?:[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+(?:-[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)?\s*){2,5}"
)

print("CORREOS:", re.findall(correo, texto))
print("FECHAS:", re.findall(fecha, texto, re.IGNORECASE))
print("FOLIOS:", re.findall(folio, texto))
print("MONTOS:", re.findall(dinero, texto))
```

Cada patrón se puede probar y corregir de forma independiente. Esta separación es más clara que una única expresión enorme y permite adaptar la extracción cuando el formato de una categoría cambia.

Normalizar errores deliberados antes de extraer

```
texto = re.sub(r"\bcorreoo\b", "correo", texto, flags=re.IGNORECASE)
```

```
texto = re.sub(r"\bdirecion\b", "dirección", texto, flags=re.IGNORECASE)
texto = re.sub(r"\bdiciplina\b", "disciplina", texto, flags=re.IGNORECASE)
texto = re.sub(r"\bcansion\b", "canción", texto, flags=re.IGNORECASE)
```

La normalización previa reduce variantes innecesarias. Después, los extractores pueden trabajar sobre un texto más uniforme.

Ejemplo breve de lematización controlada

Supongamos que se analizarán comentarios sobre compras y queremos que varias formas del verbo 'comprar' cuenten como la misma unidad conceptual.

```
import re

texto = """
El cliente compra productos.
Ayer compró una consola.
Otros clientes compraron en línea.
Estamos comprando menos este mes.
Quieren comprar mañana.
"""

patron = r"\b(?:comprar|compra|compró|compraron|comprando)\b"

formas = re.findall(patron, texto, re.IGNORECASE)

for forma in formas:
    print(f"{forma} -> comprar")
```

Salida:

```
compra -> comprar
compró -> comprar
compraron -> comprar
comprando -> comprar
comprar -> comprar
```

Este ejemplo no pretende reemplazar un lematizador de español. Su valor didáctico consiste en mostrar cómo varias superficies pueden mapearse a un mismo lema cuando el dominio y el vocabulario son controlados.

Combinar regex con One-Hot Encoding, Bag of Words y coocurrencia

Las regex pueden participar en una etapa previa de limpieza y normalización. Después, otras técnicas convierten tokens o términos en representaciones numéricas. La decisión de normalizar debe depender del objetivo: a veces conviene conservar plural, tiempo verbal o formato; otras veces conviene agruparlos.



Ejemplo con Bag of Words antes y después de normalizar

```
import re
from sklearn.feature_extraction.text import CountVectorizer

docs = [
    "cliente compra producto",
    "clientes compraron productos",
    "cliente comprando producto"
]

# Normalización controlada
docs_norm = []
for d in docs:
    d = re.sub(r"\bclientes?\b", "cliente", d)
    d = re.sub(r"\bproductos?\b", "producto", d)
    d = re.sub(r"\b(?:compra|compraron|comprando)\b", "comprar", d)
    docs_norm.append(d)

vec = CountVectorizer()
```

```
X = vec.fit_transform(docs_norm)

print(docs_norm)
print(vec.get_feature_names_out())
print(X.toarray())
```

Salida:

```
['cliente comprar producto', 'cliente comprar producto', 'cliente comprar producto']
['cliente' 'comprar' 'producto']
[[1 1 1]
 [1 1 1]
 [1 1 1]]
```

Sin normalización aparecerían columnas distintas para cliente/clientes, producto/productos y compra/compraron/comprando. En este ejemplo la intención es reunirlas; en otros problemas esas diferencias podrían ser informativas y no deberían eliminarse.

Ejemplo conceptual con One-Hot Encoding

Si el vocabulario contiene ['correo', 'correoo', 'email'], un One-Hot Encoding asignará posiciones diferentes a cada forma. Si 'correoo' es solo una falta conocida, una regla con re.sub puede convertirla en 'correo' antes de codificar. Así se evita crear una característica artificial para un error ortográfico recurrente.

Ejemplo de coocurrencia

```
import re
from collections import Counter

texto = "cliente compra producto clientes compraron productos"

texto = re.sub(r"\bclientes?\b", "cliente", texto)
texto = re.sub(r"\bproductos?\b", "producto", texto)
texto = re.sub(r"\b(?:compra|compraron)\b", "comprar", texto)

tokens = texto.split()
pares = Counter(zip(tokens, tokens[1:]))

print(tokens)
print(pares)
```

Salida:

```
['cliente', 'comprar', 'producto', 'cliente', 'comprar', 'producto']
Counter({('cliente', 'comprar'): 2, ('comprar', 'producto'): 2, ('producto', 'cliente'): 1})
```

Al normalizar, las relaciones de contexto se concentran en menos formas. Esto puede hacer más clara una matriz de coocurrencia cuando el objetivo es estudiar conceptos y no diferencias morfológicas.

Otros temas útiles para continuar

- re.IGNORECASE para ignorar mayúsculas/minúsculas cuando el caso no importa.
- re.MULTILINE para que ^ y $ puedan trabajar por línea dentro de un texto de varias líneas.
- re.DOTALL para permitir que . también abarque saltos de línea.
- re.VERBOSE para escribir patrones extensos con espacios y comentarios, lo cual mejora mucho su mantenibilidad.
- Grupos con nombre (?P<nombre>...) cuando una coincidencia contiene varias partes que se desean recuperar con etiquetas claras.
- Referencias como \1 para detectar repeticiones exactas, por ejemplo palabras duplicadas.
- Lookaheads y lookbehinds para exigir contexto sin incluirlo en la coincidencia.

Referencia rápida

Símbolo	Lectura práctica
`\b`	Frontera de palabra; no consume caracteres.
`\d / \D`	Dígito / no dígito.
`\w / \W`	Carácter de palabra / no palabra.

Símbolo	Lectura práctica
`\s / \S`	Espacio en blanco / no espacio.
.	Cualquier carácter (normalmente excepto salto de línea).
`\.`	Punto literal.
+	Una o más repeticiones.
*	Cero o más repeticiones.
?	Cero o una repetición del elemento anterior.
`{n}`	Exactamente n.
`{n,m}`	Entre n y m.
`^ / $`	Inicio / final.
|	Alternativa.
(...)	Agrupa y captura.
`(?:...)`	Agrupa sin capturar.
`(?=...)`	Condición positiva hacia adelante.
(?<=...)	Condición positiva hacia atrás.
`\1`	Repite lo capturado por el grupo 1.

Plantilla mental final. Antes de escribir una regex: 1) anote ejemplos válidos; 2) anote ejemplos que deben rechazarse; 3) marque partes fijas; 4) identifique caracteres variables; 5) decida cantidades; 6) determine qué es opcional; 7) agrupe solo donde una regla deba afectar a varias piezas; 8) pruebe el patrón y refine una condición a la vez.

Fin del apunte